Data and Reference Result

Checkpoint: November 5, 2026, by 11:59 PM Mountain Time

Team: Savanna & Jillian

Date: 11/2/2026

Use any prompts that help you describe your progress. Add a dated entry to the root README.md of your team's repository and link the relevant code and output there; this file is optional supporting material.

Data

Where did the data come from, and how did you obtain or prepare them?

Each of us exported our own transaction history as CSVs from our banks' websites: Jillian's is one Bank of America credit card (Dec 2024 to Dec 2025), and Savanna's comes from her credit and debit cards from Goldenwest, SoFi, and Capital One. The raw files stay in a gitignored `.data/` folder and are never committed. `ml/bank_statment_processor.py` converts every bank's layout into one shared format and makes signs consistent (spending negative, money in positive). We then label each transaction's main and sub category by hand using `docs/CATEGORY_GUIDE.md` (rerun: `uv sync`, then `uv run python -m ml.bank_statment_processor`).

What data are available to your team now?

We currently have Jillian's 393 Bank of America credit-card transactions all labeled, and Savanna's 998 transactions from 4 banks all labeled.

What remains to be collected or cleaned?

We need to build the made-up sample and decide how to combine both datasets privately for the cross-person test.

Route C: What does your first collected and cleaned sample contain?

Jillian's sample has 393 transactions (377 debits, 16 credits) from one credit card over 13 months, with 163 distinct merchants. Savanna's has 998 transactions (863 debits, 135 credits) across 4 banks and 7 accounts in 2025, with 423 distinct merchants, all labeled. Neither has missing values or months without transactions. Categories are unbalanced in both (Food & Dining is 62% of Jillian's and 35% of Savanna's), so we report macro-F1 alongside accuracy; full checks are in `results/data_quality_<name>.md`

Evaluation plan

What comparison, split, metric, or diagnostic will help answer your question? Why?

We will compare a most-common baseline with machine-learning models that use pretrained sentence embeddings as their shared text representation. For the first run, KNN will have two feature variants: sentence embeddings plus weighted amount, and an additional character TF-IDF-plus-amount comparison. The other planned ML models will use the sentence-embedding features rather than TF-IDF. We will use cross-validation

within Savanna's training data to compare models, reporting average scores and variation across folds. We will then evaluate the first-run KNN models on Jillian's transactions to assess performance within and across people. We will report accuracy and macro F1 separately for main categories and subcategories. Accuracy measures overall correctness, while macro F1 gives each category equal weight so that common categories do not dominate the results. Confusion matrices and separate results for familiar versus unfamiliar descriptions will help explain where the models struggle.

How will you avoid data leakage or another misleading comparison?

We will carefully compile our datasets so that the same transaction does not appear in both training and evaluation data.

The two text methods learn at different stages:

- **Sentence embeddings:** `all-MiniLM-L6-v2` is a model that was already trained on general text before this project. We use it like a calculator: we give it a transaction description, and it returns a numeric vector. We do not show it our transaction descriptions, amounts, or category labels to retrain it.
- **TF-IDF:** this vectorizer is built for our project. For each training split, it examines only the training descriptions, records which character patterns occur and how informative they are, and converts those descriptions into vectors. We then reuse that same fitted vectorizer on the test descriptions; we never refit it using the test descriptions.

In other words, the embedding model's language knowledge is pre-existing, while the TF-IDF vocabulary and weights are learned from each training split. Neither method uses category labels to create its text features. Amount scaling also uses training-set statistics only. We will select model settings through cross-validation on Savanna's training data before evaluating on held-out datasets. All models use the same evaluation splits. We will also follow a shared category guide and check labeling agreement so that differences in our preferences are not mistaken for model errors. Finally, we will distinguish performance on familiar and unfamiliar descriptions and acknowledge that results from two people do not establish performance for all users.

First runnable result

What baseline or other reference did you run? What did it produce?

For our baseline, we use a model that always predicts the most common category in the training data. The first-result runner evaluates sentence- embedding KNN with weighted amounts and a TF-IDF-plus-amount KNN comparison. Current aggregate scores are generated from private labeled CSVs and are documented in `results/first_result.md`.

Where are the code and output? Link them, and include the command needed to rerun the result if available.

The runner is `ml/first_result.py`, the model implementations are in `ml/ml_models.py`, and the output is `results/first_result.md`. Run: `uv run python -m ml.first_result --jillian <csv> --savanna <csv>`.

What did this first result suggest, and what will you try next?

Within-person, the embedding KNN improved macro-F1 over the most-common baseline for both Jillian (0.314 versus 0.101) and Savanna (0.457 versus 0.042). Its accuracy was lower than the baseline for Jillian

(0.603 versus 0.624), but higher for Savanna (0.559 versus 0.348). The TF-IDF KNN also improved on the Savanna baseline, with accuracy 0.505 and macro-F1 0.440. These results show why macro-F1 is important for these imbalanced categories: the baseline performs well on common categories while largely ignoring smaller ones.

Cross-person performance was weaker. No descriptions were shared between the two datasets. Training on Savanna and testing on Jillian produced embedding KNN accuracy of 0.496 and macro-F1 of 0.135; training on Jillian and testing on Savanna produced accuracy of 0.274 and macro-F1 of 0.089. The difference between directions reflects the different category distributions and account types, so the first result does not support assuming that a model trained on one person will transfer reliably to another.

The next steps are to inspect the largest confusion pairs, especially Food & Dining, Savings & Investments, Shopping & Supplies, and Transportation; compare additional feature and KNN settings; and evaluate the planned Random Forest and ANN/MLP models. We also need to complete the made-up reproducible sample and, if the project scope permits, evaluate subcategory predictions and labeling agreement on a shared sample. The complete scores and confusion matrices are in `results/first_result.md`, and the data-quality checks are in `results/data_quality_jillian.md` and `results/data_quality_savanna.md`.